

# **Software Requirements Specification (SRS)**

## **Notebook Intelligence System (NIS)**

Version 1.0

*Prepared as a project requirements document*

## Document Contents

1. Introduction
2. Overall Description
3. Functional Requirements
4. Non-Functional Requirements
5. System Architecture
6. Data Requirements
7. User Interface Requirements
8. Use Cases
9. Error and Exception Handling
10. Security and Privacy
11. Testing Requirements
12. Limitations and Risks
13. Future Enhancements
14. Conclusion

# 1. Introduction

## 1.1 Purpose

This document specifies the functional and non-functional requirements for the Notebook Intelligence System (NIS), an AI-assisted application that helps students digitize handwritten or technical notebook content, search it, understand it, generate quizzes, and process handwritten programming code.

## 1.2 Scope

The system will accept handwritten notebook pages as images or PDF documents. It will extract text from the pages, retain page-level association, support search and highlighting, answer questions and explain topics using the uploaded notes as context, generate quizzes from the notes, and provide a workflow for extracting and executing handwritten programming code. The system is intended as an educational assistant; OCR and code extraction may contain errors and therefore extracted code should be reviewable before execution.

## 1.3 Intended Users

Primary users are students who maintain handwritten academic notes. Secondary users may include teachers or project evaluators who want to demonstrate document understanding and learning-assistance features.

## 1.4 Definitions and Acronyms

NIS: Notebook Intelligence System. OCR: Optical Character Recognition. AI: Artificial Intelligence. LLM: Large Language Model. PDF: Portable Document Format. UI: User Interface. API: Application Programming Interface.

# 2. Overall Description

## 2.1 Product Perspective

NIS is a document-intelligence and learning-assistance application. A typical workflow is: upload notebook → organize pages → extract text → index content → search or ask questions → explain/generate quizzes. A separate code-processing workflow extracts handwritten code, presents the extracted code for review, and executes it in a controlled environment.

## 2.2 Product Goals

The main goals are to reduce the effort required to work with handwritten notes, make notebook content searchable, provide explanations grounded in the student's own notes, support revision through quizzes, and demonstrate practical multimodal AI capabilities.

## 2.3 Assumptions

Users provide reasonably legible images or PDFs; pages have sufficient resolution and contrast; the selected programming language is supported by the execution environment; the system has access to an OCR/model service when required.

## 2.4 Constraints

Handwriting recognition accuracy varies with handwriting style, image quality, mathematical notation, diagrams, and code formatting. Code execution must be sandboxed. Large documents may require processing time and storage. AI-generated explanations and quizzes may occasionally be incorrect and should be treated as assistance rather than authoritative academic material.

## 2.5 Out of Scope for the Initial Version

Perfect recognition of all handwriting styles; complete understanding of arbitrary diagrams; execution of arbitrary programming languages; unrestricted execution of uploaded code; guaranteed 100% correct OCR, explanations, quiz answers, or program output.

### 3. Functional Requirements

#### FR-01 User Upload

The system shall allow a user to upload one or more notebook images and/or PDF files.

#### FR-02 File Validation

The system shall validate file type, size, readability, and supported formats before processing.

#### FR-03 Multi-page Processing

The system shall preserve the order of uploaded pages and associate extracted content with its source page. The user may specify a starting and ending page/range when processing a selected sequence.

#### FR-04 Page Preview

The system shall display uploaded pages and allow the user to navigate between pages.

#### FR-05 Text Extraction

The system shall extract machine-readable text from handwritten or technical notes using an OCR/vision model pipeline.

#### FR-06 Extraction Review

The system should allow users to view or correct extracted text before relying on it for downstream tasks.

#### FR-07 Content Indexing

The system shall index extracted content page-wise so that words and phrases can be searched across the notebook.

#### FR-08 Search

The system shall allow the user to enter a word or phrase and search across the processed notebook.

#### FR-09 Search Highlighting

The system shall display matching page(s), relevant sentence/context, and highlight the matching word or phrase where possible.

#### FR-10 Topic Explanation

The system shall allow the user to ask a question or request an explanation of a topic found in the uploaded notes.

#### FR-11 Grounded Answers

The system should use the uploaded notebook content as context and identify relevant page(s) for an explanation where possible.

#### FR-12 Quiz Generation

The system shall generate quizzes from selected notebook content. Supported question types may include MCQs, true/false, fill-in-the-blank, and short-answer questions.

#### FR-13 Quiz Results

The system shall allow the user to submit answers and view the generated answer key and score.

#### FR-14 Code Detection

The system should identify when an uploaded page appears to contain programming code and offer the code-processing workflow.

### FR-15 Code Extraction

The system shall extract code from a handwritten program and display the extracted code to the user.

### FR-16 Code Review

The user shall be able to edit/correct extracted code before execution.

### FR-17 Code Execution

The system shall execute supported code in a controlled sandbox and display standard output and relevant execution errors.

### FR-18 Error Handling

The system shall provide understandable error messages for unsupported files, failed extraction, search failures, invalid code, and execution failures.

### FR-19 History

The system should maintain a session-level history of uploaded notebooks, searches, explanations, quizzes, and code-processing results when enabled.

### FR-20 Deletion

The system shall provide a way to remove uploaded documents and associated processed content from the active user workspace.

#### Must Have

File upload (FR-01, FR-02)

Page processing and preview (FR-03, FR-04)

OCR text extraction (FR-05)

Search (FR-08, FR-09)

Basic error handling (FR-18)

#### Should Have

Extraction review/correction (FR-06)

Topic explanation / Q&A (FR-10, FR-11)

Quiz generation and scoring (FR-12, FR-13)

Document deletion (FR-20)

#### Could Have

Session history (FR-19)

Code detection (FR-14)

Content indexing improvements

#### Won't Have (v1)

Handwritten code extraction (FR-15)

Code review and editing (FR-16)

Sandboxed code execution (FR-17)

## 4. Non-Functional Requirements

### NFR-01 Usability

The interface shall be simple enough for students with basic computer/mobile skills and shall provide clear navigation between upload, notebook view, search, explanation, quiz, and code features.

### NFR-02 Performance

For normal-sized documents, the system should provide visible processing progress and avoid appearing frozen during OCR or AI processing.

### NFR-03 Scalability

The architecture should allow additional pages, notebooks, users, and model providers to be added without redesigning the complete application.

### NFR-04 Reliability

Failures in one page should not unnecessarily discard successfully processed pages from the same upload.

### NFR-05 Security

Uploaded documents and extracted content shall be protected against unauthorized access. File type and size validation shall be applied.

### NFR-06 Code Safety

User-submitted code shall run only in an isolated sandbox with resource limits and without unrestricted access to the host system, filesystem, network, or credentials.

### NFR-07 Privacy

The system shall clearly communicate how uploaded notes are stored and processed and shall provide deletion controls.

### NFR-08 Maintainability

The application should use modular components so OCR, search, AI generation, and code execution can be replaced or upgraded independently.

### NFR-09 Accessibility

The interface should provide readable text, clear controls, keyboard-accessible navigation where applicable, and understandable error messages.

### NFR-10 Accuracy Transparency

The UI shall communicate that handwriting extraction and AI-generated content may contain errors and should be verified.

## 5. System Architecture

### 5.1 High-Level Components

1) Frontend/UI; 2) Upload and document manager; 3) Page rendering and ordering module; 4) OCR/vision processing module; 5) Text cleaning and page-wise indexing module; 6) Search and highlighting module; 7) Retrieval/AI explanation module; 8) Quiz generation module; 9) Code extraction and validation module; 10) Sandboxed code execution service; 11) Database/storage layer.

### 5.2 Suggested Data Flow

User uploads files → file validation → page extraction/rendering → OCR/vision processing → text cleaning → page-wise storage/indexing. Search requests use the index. Explanation and quiz requests retrieve relevant notebook content and pass it as context to the AI generation component. Handwritten code follows extraction → user review → validation → sandbox execution → output/error display.

### 5.3 Suggested Technology Stack

Frontend: React or another modern web framework. Backend: Python with FastAPI/Flask or equivalent. OCR/vision: a suitable handwriting-capable OCR or multimodal vision model. AI: an LLM with retrieval-augmented prompting. Search: PostgreSQL full-text search, Elasticsearch/OpenSearch, or a lightweight local index for an MVP. Storage: object/file storage for uploads and a relational database for metadata. Code execution: isolated containers or a dedicated sandbox service.

## 6. Data Requirements

### 6.1 Core Entities

User/Session, Notebook, Document, Page, ExtractedText, SearchQuery, AIExplanation, Quiz, QuizQuestion, CodeSubmission, ExecutionResult.

### 6.2 Page Record

A page record should contain a notebook/document identifier, page number/order, original file reference, rendered image reference, extracted text, processing status, and optional confidence/error information.

### 6.3 Quiz Record

A quiz should store its source notebook/pages, question type, question text, options where applicable, correct answer, explanation if generated, and user response/score where applicable.

## 7. User Interface Requirements

### 7.1 Main Screens

Dashboard; Upload Notebook; Page Viewer; Search; Ask/Explain; Quiz; Code Lab; Processing/History; Settings/Privacy.

### 7.2 Upload Screen

The upload screen shall support one or more files, show selected files, provide page-order controls, and display processing progress.

### 7.3 Notebook Viewer

The viewer should show the original page alongside extracted text where screen size permits and provide previous/next page navigation.

## 7.4 Search Screen

The search screen shall show matching pages and context. Selecting a result should open the relevant page and highlight the occurrence when technically possible.

## 7.5 Code Lab

The Code Lab shall show the original image, extracted code, an editable code area, a run action, and an output/error area.

# 8. Use Cases

## UC-01 Upload Notebook

Actor: Student. Precondition: User has a supported image/PDF. Main flow: select file(s) → validate → process pages → display status → make notebook available.

## UC-02 Search Notebook

Actor: Student. Precondition: Notebook has processed text. Main flow: enter word/phrase → search index → show matching pages/context → highlight occurrence.

## UC-03 Explain Topic

Actor: Student. Precondition: Relevant notebook content exists. Main flow: enter question/topic → retrieve relevant content → generate grounded explanation → show source page references.

## UC-04 Generate Quiz

Actor: Student. Precondition: Notebook content exists. Main flow: select pages/topic → choose quiz type/count → generate quiz → answer → show result.

## UC-05 Run Handwritten Program

Actor: Student. Precondition: Supported programming language and readable code. Main flow: upload page(s) → extract code → user reviews/edits → run in sandbox → show output/errors.

# 9. Error and Exception Handling

## Examples

Unsupported file → show supported formats. Poor image quality → request clearer image. OCR failure → mark page as failed and allow retry. No search match → state that no match was found. AI retrieval failure → show a processing error and preserve the notebook. Invalid code → show validation/runtime error without claiming successful output. Sandbox timeout/resource limit → terminate execution and report the limit.

# 10. Security and Privacy

## Requirements

Validate uploads and reject dangerous or unsupported file types. Enforce upload size limits. Authenticate users if accounts are implemented. Use access controls for stored documents. Encrypt data in transit and protect stored data according to deployment requirements. Do not expose API keys to the frontend. Run uploaded code in an isolated environment with CPU, memory, time, filesystem, and network restrictions. Provide deletion of user documents and derived data where applicable.

## 11. Testing Requirements

### Test Categories

Unit testing for text processing, search, page mapping, quiz formatting, and code validation. Integration testing for upload → OCR → storage → search and upload → OCR → AI workflows. UI testing for navigation and page highlighting. Security testing for malicious/unsupported uploads and sandbox isolation. Accuracy evaluation using a representative set of handwritten notes and handwritten programs. Performance testing with multi-page notebooks.

### Acceptance Examples

A processed page can be opened and its extracted text is associated with the correct page. Searching a word returns the correct page when that word was successfully extracted. A generated explanation is based on retrieved notebook content. A quiz is generated from selected content. Extracted code can be corrected before execution. Unsafe or unsupported code cannot access the host environment.

## 12. Limitations and Risks

### Known Risks

Handwriting variation can reduce OCR accuracy. Mathematical symbols, diagrams, tables, and mixed handwriting/printing may be difficult to extract accurately. AI-generated explanations or quiz questions can contain mistakes. Handwritten code may be extracted incorrectly, so direct automatic execution without review is unsafe. Large notebooks can increase processing cost and latency.

### Mitigation

Show original page alongside extracted content; allow corrections; keep page references; use retrieval from the uploaded notebook; provide confidence/error indicators where available; require user review before code execution; sandbox code; provide retry and manual correction options.

## 13. Future Enhancements

### Possible Extensions

Automatic page-number recognition and ordering; better mathematical equation recognition; diagram understanding; multilingual handwriting support; personalized revision plans; spaced-repetition flashcards; teacher/classroom sharing; collaborative notebooks; offline processing for selected features; support for additional programming languages; citation-style source references to exact pages.

## 14. Conclusion

### Summary

The Notebook Intelligence System combines document processing, multimodal AI, search, educational assistance, and controlled code execution in one student-focused application. The recommended development approach is incremental: establish reliable page processing and search first, then add grounded explanation and quiz generation, followed by the more technically demanding handwritten-code workflow.